

Group Members

1. Lim Chiew Ting
2. Loo Jie Qi
3. Ng Shen Zhi
4. Tan Huei Qing
5. Choo Jing Chuen

1. Recall the principles of software testing.
 1. Testing shows the presence of defects, not their absence
 2. Exhaustive testing is impossible
 3. Testing activities should start as early as possible
 4. Defect clustering
 5. The pesticide paradox
 6. Testing is context dependent
 7. No failures means the system is useful, is a fallacy
2. Software testing is unavoidable when developing the software. Some said software testing is an “umbrella activity”. Analyse the reason behind.

Umbrella activities provide essential support for the main development tasks. They ensure project management, quality assurance, and configuration control, maintaining consistency and quality. Documentation supports understanding and future maintenance. Overall, these activities enhance efficiency and reliability throughout the development process.
3. Early testing is one of the principles of software testing, throughout the software development life cycle, identify what artifacts will be tested after the Analysis, Design, Coding and Implementation stage.

Phases	Tested Artifacts	Explanation
Analysis	Requirement Specifications	checked for completeness and clarity
Design	System Architecture & Design Documents	verified for correctness and feasibility
Coding	Source Code & Unit Tests	tested for functionality and errors
Implementation	Deployed System & User Acceptance Testing (UAT)	ensured it meets business needs

4. Why is testing important?
Testing is important because it ensures software quality, detects bugs, enhances security, improves performance, and verifies that the system meets user requirements. It reduces risks and ensures a smooth user experience.
5. Different systems cannot have the same way of testing? Do you agree? Justify your points.
Different systems may have unique requirements, but testing principles like unit, integration, and system testing apply universally. Standard frameworks, reusable tools, and automation techniques ensure consistency, though testing depth varies based on system complexity and security needs.
6. Explain the relationship between **Error**, **Defect** and **Failure**.
An **error** is a mistake made during development, which leads to a **defect** in the software. If the defect is not fixed, it results in a **failure** when the software is run.
7. Why some defects in the software will not turn into failure. Explain with one example.
Some defects in software may not result in failures because they don't affect the software's core functionality or occur in scenarios that are rarely encountered.

Example: A minor visual glitch in a non-critical part of the application, like a misaligned button on a rarely used settings page, may be a defect, but it won't cause a failure because it doesn't impact the software's primary functionality or user experience in most cases.

8. When performing testing, some tests have a higher priority than others. What are the criteria to be considered in prioritizing the tests?
 1. **Risk and Impact:** Tests that cover critical functionality or high-risk areas should be prioritized. For example, core features of the application or security-sensitive components.
 2. **Frequency of Use:** Features that are used more frequently by users should be tested first to ensure the most important aspects of the software work correctly.
 3. **Complexity:** Tests for more complex features or components, which are prone to errors, should be prioritized over simpler, less error-prone areas.
 4. **Business Value:** Features that deliver the most value to the business or the end user should be tested first to ensure that high-priority functionalities are working.
 5. **Regulatory or Compliance Requirements:** If the software needs to meet certain legal or compliance standards, tests related to these requirements should be given higher priority.
 6. **Test Dependencies:** Some tests depend on others being executed first (e.g., system tests may depend on successful integration tests), so those foundational tests should be prioritized.

7. **Past Issues:** Areas that have had defects in the past or have historically been prone to bugs should be prioritized to ensure issues don't reoccur.